

The Krasnosel'skii-Mann Iterative Method

Chapter 1: Introduction

Based on Dong, Cho, He, Pardalos & Rassias

SpringerBriefs in Optimization, 2022

Chapter Overview

- 1 Notation you'll need
- 2 The fixed point problem
- 3 1.1 Fixed Point Iteration Procedures
- 4 1.2 Fixed Point Formulation of Typical Problems
- 5 1.3 Outline
- 6 Summary

Notation and Background You Need I

Before diving in, here is a plain-English glossary of every non-elementary term used later in this deck. **We will use $\mathbb{R}^2$ as the concrete setting for every example**, so nothing here needs to stay abstract.

- **Hilbert space $\mathcal{H}$** : think of $\mathbb{R}^n$ generalized to possibly infinite dimensions, but keeping the same notion of length and angle. Concretely in this deck, $\mathcal{H} = \mathbb{R}^2$ with the usual dot product.
- **Inner product $\langle x, y \rangle$** : the ordinary dot product $x^{(1)}y^{(1)} + x^{(2)}y^{(2)}$ in $\mathbb{R}^2$; measures “how much two vectors point the same way.”
- **Norm $\|x\|$** : the length of the vector x , $\|x\| = \sqrt{\langle x, x \rangle}$ (ordinary Euclidean length in $\mathbb{R}^2$).
- **Closed convex set C** : a set with no gaps at its boundary (closed) such that the straight segment between any two of its points stays inside it (convex). Running example: the closed unit disk $C = \{x \in \mathbb{R}^2 : \|x\| \leq 1\}$.

Notation and Background You Need II

- **Operator / mapping** $T : C \rightarrow C$: just a function that takes a point of C and returns another point of C .
- **Fixed point of T** : a point x that T sends to itself, $T(x) = x$ – i.e., a point the map “leaves alone.” $\text{Fix}(T)$ denotes the set of all fixed points of T .
- **Nonexpansive operator**: a map that never increases the distance between two points, $\|T(x) - T(y)\| \leq \|x - y\|$ for all x, y – informally, “non-stretching.” It is allowed to preserve distances exactly (like a rotation), so it need not pull points together.
- **Contraction**: a *strictly* shrinking map: there is a fixed constant $\alpha \in [0, 1)$ with $\|T(x) - T(y)\| \leq \alpha \|x - y\|$ for all x, y . Every contraction is nonexpansive, but not conversely (a rotation is nonexpansive but not a contraction).
- **Firmly nonexpansive operator**: an even more well-behaved special case of nonexpansive, satisfying $\|T(x) - T(y)\|^2 \leq \langle T(x) - T(y), x - y \rangle$. The metric projection P_C (below) is always firmly nonexpansive.

Notation and Background You Need III

- **Metric projection** $P_C(x)$: the closest point of C to x . In our running example ($C =$ closed unit disk),

$$P_C(x) = \begin{cases} x, & \|x\| \leq 1, \\ x/\|x\|, & \|x\| > 1. \end{cases}$$

Geometrically: if x is already inside the disk, leave it alone; otherwise, slide it straight in to the nearest boundary point.

- **Banach contraction principle**: if T is a contraction on a complete space, it has exactly one fixed point, and repeatedly applying T from any starting point converges to it. This is the classical engine behind Picard iteration (Section 1.1) – but it needs *strict* shrinking, which nonexpansive maps do not guarantee.
- **Strong vs. weak convergence (informal)**: in $\mathbb{R}^n$ these coincide, so do not worry about the distinction while thinking in $\mathbb{R}^2$; in infinite dimensions they can differ, and the book flags which kind of convergence each theorem gives.

Why Fixed Points? The Central Problem

Many computational questions – solving equations, finding equilibria, minimizing a function – can be rephrased as: *is there a point that a certain map sends to itself, and how do we find it?* This chapter sets up that single unifying question.

The fixed point problem (Eq. 1.1)

Let $\mathcal{H}$ be a Hilbert space (reminder: think $\mathbb{R}^n$, generalized), C a nonempty closed convex subset of $\mathcal{H}$, and $T : C \rightarrow C$ an operator. Denote by $\text{Fix}(T) = \{x \in C : x = T(x)\}$ the set of fixed points of T .

Find $x^* \in C$ such that $T(x^*) = x^*$.

- $\mathcal{H}$: the ambient space (here, always $\mathbb{R}^2$).
- C : the region we search in (here, the closed unit disk).
- T : the map whose fixed points we seek (here, $T = P_C$).

This problem has applications in dynamical systems, integral and differential equations, mathematical economics (game theory, equilibrium problems), and optimization. Three natural questions arise: (a) does a fixed point exist? (b) is it unique? (c) if it exists, how do we *compute* it? This book – and this chapter – focus on (c).

Existence and Uniqueness: What Can Go Wrong I

Before building algorithms, it helps to see how differently contractions and nonexpansive maps behave regarding existence/uniqueness of fixed points – this motivates why an entire book is devoted to the nonexpansive case.

- If $T : \mathcal{H} \rightarrow \mathcal{H}$ is a *contraction* (reminder: strictly shrinking by a fixed factor $\alpha < 1$), the Banach contraction principle guarantees *exactly one* fixed point.
- If T is merely *nonexpansive* (reminder: non-stretching, but not necessarily shrinking), the situation is qualitatively different:
 - T may have **no** fixed point at all: e.g. $T(x) = x + z$ for a fixed nonzero vector z (a pure translation) is nonexpansive ($\|T(x) - T(y)\| = \|x - y\|$ always) but never sends any point to itself.
 - T may have **exactly one** fixed point: e.g. $T = -\text{Id}$ (send $x \mapsto -x$) is nonexpansive, and its only fixed point is the origin.
 - T may have **infinitely many**: e.g. $T = \text{Id}$ (the identity) fixes every point.
 - Even *firmly* nonexpansive operators can fail to have a fixed point in general infinite-dimensional settings (a subtlety we will not need in $\mathbb{R}^2$, since our running example's $T = P_C$ always has fixed points – namely all of C itself).

Existence and Uniqueness: What Can Go Wrong II

- **Consequence:** because contractions behave so well (existence *and* uniqueness, plus a convergent algorithm for free), most of the difficulty – and most of this book – concerns the *merely nonexpansive* case, where we must work harder both to guarantee convergence and to design the right iterative scheme.

1.1(A) The Picard Iteration I

The simplest possible idea: just keep applying T over and over, and hope the results settle down. This is the Picard iteration.

Picard iteration (Eq. 1.2)

Let C be a nonempty closed convex set of $\mathcal{H}$ and $T : C \rightarrow C$ a mapping. For an arbitrary $x_0 \in C$, the *Picard iteration* $\{x_n\}$ is

$$x_{n+1} = T(x_n) \quad \text{for each } n \geq 0.$$

- x_0 : any chosen starting point in C .
- $x_{n+1} = T(x_n)$: repeatedly “re-apply T ” to the last point.

By the Banach contraction principle (notation frame), Picard iteration **converges to the unique fixed point whenever T is a contraction** – and one even gets explicit error bounds telling you when to stop.

1.1(A) The Picard Iteration II

The catch: Picard iteration may fail to produce a fixed point when T is only nonexpansive (not a contraction). A simple illustration: $T = -\text{Id}$ (so $T(x) = -x$) and $x_0 \neq 0$.

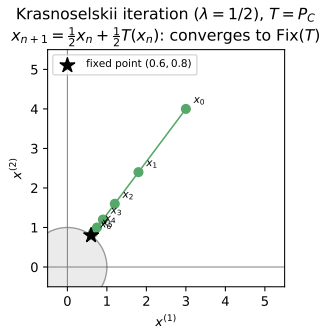
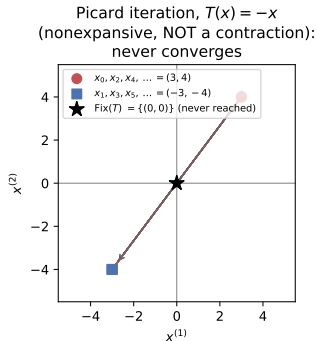
Worked failure example: $T(x) = -x$, $x_0 = (3, 4)$

- **Step 1** (apply T once): $x_1 = T(x_0) = -(3, 4) = (-3, -4)$.
- **Step 2** (apply T again): $x_2 = T(x_1) = -(-3, -4) = (3, 4) = x_0$.
- **Conclusion:** the sequence just oscillates $(3, 4), (-3, -4), (3, 4), (-3, -4), \dots$ forever – it never settles down, even though T has a (unique) fixed point at the origin ($\text{Fix}(T) = \{(0, 0)\}$).

This is exactly the left panel of the figure below.

1.1(A) The Picard Iteration III

Running example: Picard can fail for nonexpansive maps, but averaging (Krasnoselskii) fixes it



Left: Picard iteration on the nonexpansive (non-contractive) map $T(x) = -x$ started at $x_0 = (3, 4)$: it oscillates forever and never reaches the fixed point $(0, 0)$. **Right:** the Krasnosel'skiĭ iteration (next slide) on our running example $T = P_C$, started at the same-shaped point $x_0 = (3, 4)$, converges smoothly to the fixed point $(0.6, 0.8)$.

1.1(B) The Krasnosel'skiĭ Iteration I

To rescue Picard iteration for nonexpansive maps, Krasnosel'skiĭ (1955) proposed *damping*: instead of jumping all the way to $T(x_n)$, move only *halfway* there. Intuition: averaging the current point with its image under T prevents the oscillation we just saw.

Krasnosel'skiĭ iteration (Eq. 1.3)

$$x_{n+1} = \frac{1}{2}x_n + \frac{1}{2}T(x_n) \quad \text{for each } n \geq 0,$$

where $x_0 \in C$ and $T : C \rightarrow C$ is a mapping.

- Each new point is the *midpoint* of the current point x_n and its image $T(x_n)$ – a “50/50 blend” of “stay put” and “jump to $T(x_n)$.”

1.1(B) The Krasnosel'skiĭ Iteration II

Theorem 1.1 (Krasnosel'skiĭ)

Let X be a uniformly convex Banach space and C a convex compact subset of X . If $T : C \rightarrow C$ is a nonexpansive mapping with $\text{Fix}(T) \neq \emptyset$, then $\{x_n\}$ defined by Eq. 1.3 converges strongly to a fixed point of T .

1.1(B) The Krasnosel'skiĭ Iteration III

Edelstein later weakened “uniformly convex” to “strictly convex.” Schaefer extended the scheme by replacing the fixed weight $\frac{1}{2}$ with any constant $\lambda \in (0, 1)$:

General Krasnosel'skiĭ iteration (Eq. 1.4)

$$x_{n+1} = (1 - \lambda)x_n + \lambda T(x_n) \quad \text{for each } n \geq 0.$$

- $\lambda \in (0, 1)$: the (constant) relaxation/damping parameter; $\lambda = 1$ recovers plain Picard iteration exactly.

Schaefer proved weak convergence of Eq. 1.4 for a class of continuous nonexpansive operators. This iteration is exactly the Picard iteration applied to the *averaged operator*

$$T_\lambda = (1 - \lambda)\text{Id} + \lambda T,$$

and one has $\text{Fix}(T) = \text{Fix}(T_\lambda)$ for every $\lambda \in (0, 1]$ – i.e., averaging never creates or destroys fixed points, it only changes how fast (and whether) Picard iteration on T_λ converges to one.

Worked Example: Krasnosel'skii on Our Running Example I

Setup (running example, reused throughout this book's chapters). $\mathcal{H} = \mathbb{R}^2$, $C = \{x : \|x\| \leq 1\}$ (closed unit disk), $T = P_C$ (metric projection; reminder from the notation frame). Start at $x_0 = (3, 4)$, and use $\lambda = \frac{1}{2}$ (Eq. 1.3).

Key observation: everything stays on one ray

Write $x_0 = 5 \cdot (0.6, 0.8)$ – note $(0.6, 0.8)$ has norm exactly 1 (it's already a boundary point of C , hence already a fixed point of T). Since T only rescales vectors (it never changes their direction), every iterate stays on the ray $\{r \cdot (0.6, 0.8) : r \geq 0\}$. So we can track just the scalar r_n , where $x_n = r_n \cdot (0.6, 0.8)$.

As long as $r_n > 1$ (point outside the disk), $T(x_n) = x_n / \|x_n\| = (0.6, 0.8)$ exactly (norm-1 point), so the recursion collapses to

$$r_{n+1} = \frac{1}{2}r_n + \frac{1}{2} \cdot 1.$$

Worked Example: Krasnosel'skii on Our Running Example II

Step-by-step computation, $r_0 = 5$

- **Step 1:** $r_1 = \frac{1}{2}(5) + \frac{1}{2}(1) = 2.5 + 0.5 = 3.$
- **Step 2:** $r_2 = \frac{1}{2}(3) + \frac{1}{2}(1) = 1.5 + 0.5 = 2.$
- **Step 3:** $r_3 = \frac{1}{2}(2) + \frac{1}{2}(1) = 1 + 0.5 = 1.5.$
- **Step 4:** $r_4 = \frac{1}{2}(1.5) + \frac{1}{2}(1) = 0.75 + 0.5 = 1.25.$
- **Step 5:** $r_5 = \frac{1}{2}(1.25) + \frac{1}{2}(1) = 0.625 + 0.5 = 1.125.$
- **Step 6:** $r_6 = \frac{1}{2}(1.125) + \frac{1}{2}(1) = 0.5625 + 0.5 = 1.0625.$
- **Pattern:** $r_n - 1 = (r_0 - 1) \left(\frac{1}{2}\right)^n = 4 \cdot \left(\frac{1}{2}\right)^n \rightarrow 0$, so $r_n \rightarrow 1$ *geometrically*.

Conclusion: $x_n = r_n \cdot (0.6, 0.8) \rightarrow (0.6, 0.8)$, the unique fixed point of T on this ray – exactly the right panel of the figure on the previous frame. Contrast with $T = -\text{Id}$: plain Picard ($\lambda = 1$) never converges, but damping with $\lambda < 1$ fixes this completely (in fact, applying Eq. 1.3 to $T = -\text{Id}$ from $x_0 = (3, 4)$ gives $x_1 = \frac{1}{2}(3, 4) + \frac{1}{2}(-3, -4) = (0, 0)$ – the fixed point, in a single step!).

Python: Picard Failure vs. Krasnosel'skii Success

```
1 import numpy as np
2 def project_disk(x):                                #  $T = P_C$ 
3     n = np.linalg.norm(x)
4     return x if n <= 1.0 else x / n
5
6 x0 = np.array([3.0, 4.0])
7 x = x0.copy(); picard_trace = [x.copy()]           # Picard on  $T(x) = -x$ 
8 for _ in range(4):
9     x = -x
10    picard_trace.append(x.copy())
11 print("Picard on T=-Id:", [tuple(p) for p in picard_trace])
12
13 lam = 0.5
14 x = x0.copy(); km_trace = [x.copy()]               # Krasnoselskii on  $T = P_C$ 
15 for _ in range(6):
16     x = (1 - lam) * x + lam * project_disk(x)      # Eq. 1.3
17     km_trace.append(x.copy())
18 for n, p in enumerate(km_trace):
19     print(f"r_{n} = {np.linalg.norm(p):.4f}    x_{n} = ({p[0]:.4f}, {p[1]:.4f})")
```

Output (excerpt): Picard on T=-Id: [(3,4), (-3,-4), (3,4), (-3,-4), (3,4)] (never converges), and r_0=5.0000 ... r_6=1.0625, matching the hand computation exactly.

1.1(B') The Krasnosel'skii–Mann (“Mann”) Iteration I

The constant weight λ can itself be replaced by a whole *sequence* λ_n , allowed to change at every step. This most general averaging scheme is due to Mann, and the resulting family of methods is now called the Krasnosel'skii–Mann iteration.

Krasnosel'skii–Mann iteration (Eq. 1.5)

$$x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T(x_n) \quad \text{for each } n \geq 0,$$

where $\{\lambda_n\} \subset [0, 1]$.

- λ_n : the relaxation parameter *at step* n – no longer required to be constant. Choosing $\lambda_n \equiv \lambda$ recovers Eq. 1.4; choosing $\lambda_n \equiv 1$ recovers Picard.

This scheme (Eq. 1.5) has been studied in a great many papers over the last sixty years. It is the central object of Chapter 3 of this book.

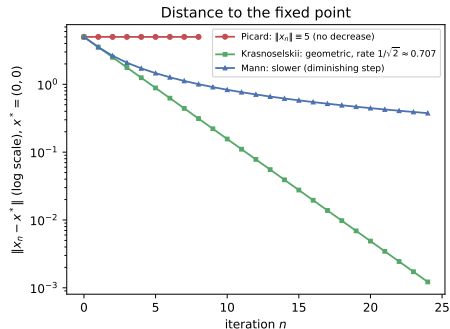
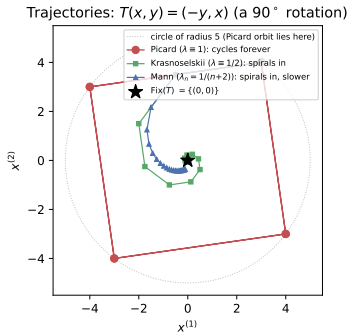
1.1(B') The Krasnosel'skiĭ–Mann (“Mann”) Iteration II

Illustration with our rotation toy example. To compare Picard, constant- λ Krasnosel'skiĭ, and diminishing-step Mann *on the same simple problem* (our disk-projection $T = P_C$ makes Picard converge trivially in one step, so it doesn't showcase the contrast), use instead $T(x, y) = (-y, x)$ – a 90° rotation in $\mathbb{R}^2$. It is nonexpansive (a rotation never changes lengths) with unique fixed point $\text{Fix}(T) = \{(0, 0)\}$.

- **Picard** ($\lambda_n \equiv 1$): from $x_0 = (3, 4)$, iterates cycle around the circle of radius 5 forever – $(3, 4) \rightarrow (-4, 3) \rightarrow (-3, -4) \rightarrow (4, -3) \rightarrow (3, 4) \rightarrow \dots$ – never converging.
- **Krasnosel'skiĭ** ($\lambda_n \equiv \frac{1}{2}$): spirals into the origin *geometrically*, shrinking by a factor $1/\sqrt{2} \approx 0.707$ every step (the fastest possible rate for this example, as shown two slides ahead).
- **Mann** ($\lambda_n = 1/(n+2) \rightarrow 0$): also spirals into the origin, but more slowly, since the damping weakens as n grows.

1.1(B') The Krasnosel'skii–Mann (“Mann”) Iteration III

Same nonexpansive map, three iteration schemes: damping is what makes convergence possible



Same nonexpansive map $T(x, y) = (-y, x)$, three schemes, same start $x_0 = (3, 4)$. **Left:** trajectories in the plane. **Right:** distance to the fixed point $(0, 0)$ versus iteration number (log scale) – Picard never shrinks, Krasnosel'skii shrinks geometrically fastest, Mann's diminishing steps shrink more slowly but still converge.

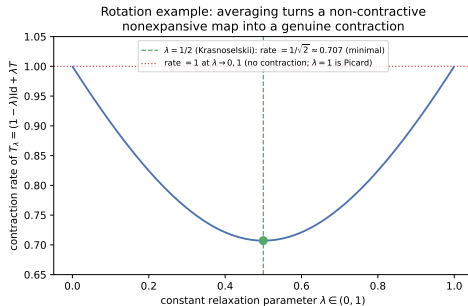
Why $\lambda = 1/2$ Is Special Here: Rate vs. Relaxation Parameter

For the rotation example, one can compute exactly how fast the constant- λ averaged map $T_\lambda = (1 - \lambda)\text{Id} + \lambda T$ contracts, as a function of λ .

Contraction rate of T_λ for a 90° rotation

Representing $\mathbb{R}^2$ as the complex plane ($T = \text{multiplication by } i$), T_λ multiplies by $(1 - \lambda) + \lambda i$, whose modulus is the contraction rate: $\text{rate}(\lambda) = \sqrt{(1 - \lambda)^2 + \lambda^2}$.

- At $\lambda = 1$ (Picard): rate = 1 – no contraction at all, consistent with the never-ending cycle we saw.
- At $\lambda = 0$: rate = 1 too (the map never moves).
- Minimized at $\lambda = 1/2$: rate = $1/\sqrt{2} \approx 0.707$ – exactly Krasnosel'skii's original, historically first choice!



1.1(C) The Ishikawa Iteration I

Sometimes one averaging step per iteration is not enough to guarantee convergence for harder classes of maps (beyond nonexpansive). The Ishikawa iteration adds a *second*, inner averaging step before the outer update.

Ishikawa iteration (Eq. 1.6)

For an arbitrary $x_0 \in C$, define $\{x_n\}$ by: for each $n \geq 0$,

$$\begin{cases} y_n = (1 - \alpha_n)x_n + \alpha_n T(x_n), \\ x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T(y_n). \end{cases}$$

- y_n : an *inner* Krasnosel'skii-type averaging step, with its own parameter α_n .
- x_{n+1} : the *outer* update, applying T to y_n (instead of directly to x_n) and averaging with weight λ_n .
- This is sometimes called the *two-step* Krasnosel'skii–Mann iteration.

1.1(C) The Ishikawa Iteration II

The Ishikawa iteration was first used to establish strong convergence to a fixed point for a Lipschitz and *pseudocontractive* self-map of a convex compact subset C of a Hilbert space – a broader class of maps than nonexpansive ones, for which the plain Krasnosel'skii–Mann iteration (Eq. 1.5) is known to fail (Chidume–Mutangadura built an explicit counterexample).

1.1(C) The Ishikawa Iteration III

Trade-offs to keep in mind:

- The Ishikawa iteration is computationally *more expensive* per step than Krasnosel'skiĭ–Mann (it evaluates T at y_n , an extra averaged point).
- Its convergence *rate* is known to be *slower* than that of Krasnosel'skiĭ–Mann when both apply.
- **Practical rule of thumb:** when several iterative schemes are known to converge for your class of maps, prefer the simplest one – which is usually Krasnosel'skiĭ–Mann.

Weak vs. Strong Convergence: Why We Need Modifications

In infinite-dimensional spaces (unlike $\mathbb{R}^n$, where the two notions coincide – reminder from the notation frame), the Krasnosel'skii–Mann and Ishikawa iterations are generally only guaranteed to converge *weakly*.

- Genel and Lindenstrauss constructed a celebrated counterexample: a *contractive* operator on a bounded closed convex subset of a Hilbert space for which the Krasnosel'skii iteration does **not** converge strongly.
- Another counterexample exists for the Picard iteration applied to the *composition of two projections*.
- Yet strong convergence is often much more desirable in applications than weak convergence.

Two remedies pursued in the literature (and in this book):

- ➊ **Additional conditions on T** – e.g., if T is *demiccontractive* and $\text{Id} - T$ maps closed bounded subsets of C onto closed subsets of C (in particular if T is demicompact), then Eq. 1.5 converges *strongly*. Maruster gave another sufficient condition: existence of $h \neq 0$ with $\langle x - T(x), h \rangle \leq 0$ for all $x \in C$.
- ➋ **Modifications of the iteration itself** – the Halpern iteration and the hybrid (“CQ”) methods, covered next.

1.1(D) The Halpern Iteration I

Idea: instead of averaging x_n with $T(x_n)$, average a *fixed anchor point* u with $T(x_n)$. Intuition: u acts like a “magnet” that pulls the whole sequence toward the fixed point of T that is *closest to* u , which is exactly what buys us strong convergence.

Halpern iteration (Eq. 1.7)

For any fixed $u \in C$ and arbitrary $x_0 \in C$:

$$x_{n+1} = \lambda_n u + (1 - \lambda_n)T(x_n) \quad \text{for each } n \geq 0,$$

where $\{\lambda_n\} \subset [0, 1]$ and the convex combination is “anchored” to u .

- u : a fixed, chosen “anchor” point in C (not updated).
- λ_n : the weight on the anchor at step n (typically $\rightarrow 0$, so the anchor’s pull weakens over time but never vanishes in total, per the conditions below).

1.1(D) The Halpern Iteration II

Standard conditions on $\{\lambda_n\}$

- (C1) $\lim_{n \rightarrow \infty} \lambda_n = 0$;
- (C2) $\sum_{n=0}^{\infty} \lambda_n = \infty$;
- (C3) $\sum_{n=0}^{\infty} |\lambda_{n+1} - \lambda_n| < \infty$, or $\lim_{n \rightarrow \infty} \lambda_n / \lambda_{n+1} = 1$.

1.1(D) The Halpern Iteration III

Convergence guarantee. If $T : C \rightarrow C$ is nonexpansive with $\text{Fix}(T) \neq \emptyset$ and $\{\lambda_n\}$ satisfies (C1)–(C3), then $\{x_n\}$ from Eq. 1.7 converges *strongly* to $P_{\text{Fix}(T)}u$ – i.e., the point of $\text{Fix}(T)$ *closest to the anchor* u (reminder: P_K denotes metric projection onto K).

A tight, quantitative convergence-rate bound

With the popular choice $\lambda_n = \frac{1}{n+2}$ and $u = x_0$:

$$\|x_n - T(x_n)\| \leq \frac{2\|x_0 - x^*\|}{n+1} \quad \text{for each } n \geq 1 \text{ and } x^* \in \text{Fix}(T).$$

- Left side: how far the current iterate is from being a fixed point (a computable “residual”).
- Right side: an explicit $O(1/n)$ bound, entirely in terms of the starting distance to *any* fixed point x^* .

1.1(D) The Halpern Iteration IV

The convergence speed of the Halpern iteration is widely believed to be slow, due to the strict restriction (C2). Later work found improved, data-driven choices of $\{\lambda_n\}$ (Eq. 1.8 in the book) that speed this up substantially in practice – we omit the lengthy closed form here and refer to it only by name.

1.1(D) The Halpern Iteration V

A useful special case: minimum-norm fixed points. Taking $u = 0$ in Eq. 1.7 targets $P_{\text{Fix}(T)}(0)$, the fixed point of T closest to the origin – of independent interest in convex optimization. But if $0 \notin C$, the naive analogue

$$x_{n+1} = (1 - \lambda_n)T(x_n)$$

can leave C entirely (since it drops the anchor term), so it cannot be used directly. Two fixes appear in the literature:

- **Project back into C** after each step (Wang–Xu, Eq. 1.9): $x_{n+1} = P_C[(1 - \lambda_n)T(x_n)]$ – correct, but often hard to implement because P_C itself may have no closed form.
- **An explicit boundary-point algorithm** (He et al.): define $h(x) = \inf\{\lambda \in (0, 1] : \lambda x \in C\}$ for $x \in C$ (Eq. 1.10) – “how much must we shrink x toward the origin to land exactly on the boundary of C (through the origin’s direction)?” – and iterate

$$x_{n+1} = \lambda_n \theta_n x_n + (1 - \lambda_n)T(x_n), \quad \theta_n = h(x_n)$$

(Eq. 1.11), which converges strongly to $P_{\text{Fix}(T)}(0)$ under mild conditions on $\{\lambda_n\}, \{\theta_n\}$ (Theorem 1.2). The main practical difficulty is computing h in closed form; this is doable, e.g., when C is defined by a single quadratic inequality.

1.1(E) The Viscosity Iteration I

Moudafi extended the Halpern iteration by replacing the *fixed* anchor u with a *moving* anchor $f(x_n)$, where f is a contraction – like a “brake” riding along with the current iterate rather than a single fixed magnet.

Viscosity iteration (Eq. 1.12)

$$x_{n+1} = \lambda_n f(x_n) + (1 - \lambda_n)T(x_n) \quad \text{for each } n \geq 0,$$

where $f : C \rightarrow C$ is a contraction (reminder: strictly shrinking by a fixed factor < 1).

- f : the “viscosity” term – replaces the constant anchor u of Halpern's scheme with a contraction applied to the current iterate.
- Setting $f \equiv u$ (a constant map) recovers the Halpern iteration exactly.

1.1(E) The Viscosity Iteration II

Theorem 1.3 (X_U)

Let $T : C \rightarrow C$ be nonexpansive with $\text{Fix}(T) \neq \emptyset$ and $f : C \rightarrow C$ a contraction with constant $\alpha \in [0, 1)$. Under conditions (C1)–(C3) on $\{\lambda_n\}$, the sequence $\{x_n\}$ from Eq. 1.12 converges strongly to the unique solution $q \in \text{Fix}(T)$ of the variational inequality

$$\langle f(q) - q, x - q \rangle \leq 0 \quad \text{for all } x \in \text{Fix}(T).$$

1.1(E) The Viscosity Iteration III

- Why is this variational inequality solvable uniquely? Because $\text{Id} - f$ turns out to be $(1 + \alpha)$ -Lipschitz and $(1 - \alpha)$ -strongly monotone (formal definitions come in Chapter 2; intuitively, $\text{Id} - f$ is well-behaved enough to guarantee a unique solution).
- The introduction of the viscosity term is not just cosmetic: it lets the limit point depend on f (not only on the geometry of $\text{Fix}(T)$), which is exploited heavily in applications (e.g., to select a specific, “regularized” fixed point).
- Many further extensions and generalizations of Eq. 1.12 exist in the literature; He et al. also gave optimal choices of $\{\lambda_n\}$ for this scheme, analogous to Eq. 1.8 for Halpern’s iteration.

1.1(E') The Regularized Krasnosel'skiĭ–Mann Iteration

Yao et al. introduced a variant that regularizes the *current point* itself (rather than adding an anchor or viscosity term), by shrinking it toward the origin before applying T .

Regularized Krasnosel'skiĭ–Mann iteration (Eq. 1.14)

For $T : \mathcal{H} \rightarrow \mathcal{H}$ nonexpansive: for each $n \geq 0$,

$$\begin{cases} y_n = \beta_n x_n, \\ x_{n+1} = (1 - \lambda_n) y_n + \lambda_n T(y_n), \end{cases}$$

with $\{\lambda_n\}, \{\beta_n\} \subset [0, 1]$.

- $y_n = \beta_n x_n$: shrinks x_n toward the origin by factor β_n *before* averaging with $T(y_n)$.

Theorem 1.4 (Boţ et al., improving on Yao et al.)

If $\{\lambda_n\}, \{\beta_n\}$ satisfy: $0 < \beta_n \leq 1$, $\lim_n \beta_n = 1$, $\sum_n (1 - \beta_n) = \infty$, $\sum_n |\beta_n - \beta_{n-1}| < \infty$; and $0 < \lambda_n \leq 1$, $\liminf_n \lambda_n > 0$, $\sum_n |\lambda_n - \lambda_{n-1}| < \infty$; then $\{x_n\}$ converges *strongly* to $P_{\text{Fix}(T)}(0)$ – the minimum-norm fixed point, obtained here *without* needing anchor points or explicit projections.

1.1(F) The Hybrid Methods (the “CQ” Algorithm) I

A completely different strategy for forcing strong convergence: at each step, build two auxiliary closed convex sets C_n (“don’t retreat from the target”) and Q_n (“stay consistent with your history”), then *project the original starting point* x_0 onto their intersection. Independently proposed in Haugazeau’s unpublished dissertation, and later combined with the Krasnosel’skiĭ–Mann iteration by Nakajo and Takahashi.

1.1(F) The Hybrid Methods (the “CQ” Algorithm) II

The Nakajo–Takahashi hybrid (“CQ”) algorithm (Eq. 1.15)

$x_0 \in C$ chosen arbitrarily; for each $n \geq 0$,

$$\begin{cases} y_n = \lambda_n x_n + (1 - \lambda_n)T(x_n), \\ C_n = \{z \in C : \|y_n - z\| \leq \|x_n - z\|\}, \\ Q_n = \{z \in C : \langle x_n - z, x_n - x_0 \rangle \leq 0\}, \\ x_{n+1} = P_{C_n \cap Q_n} x_0. \end{cases}$$

- y_n : an ordinary Krasnosel'skii–Mann step from x_n .
- C_n : the set of points *at least as close to y_n as x_n is* – a “don't get farther from the fixed point candidate” constraint.
- Q_n : a half-space built from the history of the iteration, separating x_0 from “already-explored” territory.
- x_{n+1} : re-project the *original* x_0 onto $C_n \cap Q_n$ – this is what is often called the *CQ algorithm*.

1.1(F) The Hybrid Methods (the “CQ” Algorithm) III

Convergence. If $T : C \rightarrow C$ is nonexpansive with $\text{Fix}(T) \neq \emptyset$ and $\{\lambda_n\} \subset [0, \sigma]$ for some $\sigma \in [0, 1)$, then $x_n \rightarrow P_{\text{Fix}(T)}(x_0)$ *strongly* – under strictly weaker conditions on $\{\lambda_n\}$ than the Halpern iteration needs.

A family of further refinements. Since Eq. 1.15, many modifications have been proposed; the book highlights three:

- **Shrinking projection method** (Eq. 1.16): start with $C_1 = C$, $x_1 = P_{C_1}x_0$, and for $n \geq 0$,

$$y_n = \lambda_n x_n + (1 - \lambda_n)T_n(x_n), \quad C_{n+1} = \{z \in C_n : \|y_n - z\| \leq \|x_n - z\|\}, \quad x_{n+1} = P_{C_{n+1}}x_0.$$

Here C_n *shrinks* monotonically (a half-space is intersected in at every step), which limits practical use for large n .

1.1(F) The Hybrid Methods (the “CQ” Algorithm) IV

- **An efficient numerical variant** (Eq. 1.17): with $x_0, z_0 \in C$ arbitrary and $\lambda_n \in [0, \sigma]$, $\sigma \in [0, \frac{1}{2})$,

$$z_{n+1} = \lambda_n z_n + (1 - \lambda_n)T(x_n), \quad C_n = \{z \in C : \|z_{n+1} - z\|^2 \leq \alpha_n \|z_n - z\|^2 + (1 - \alpha_n)\|x_n - z\|^2\},$$

with Q_n exactly as in Eq. 1.15 and $x_{n+1} = P_{C_n \cap Q_n} x_0$.

- **The main practical bottleneck:** computing $x_{n+1} = P_{C_n \cap Q_n} x_0$ is, in general, itself a nontrivial optimization problem – despite many papers studying hybrid methods theoretically, very few give a way to actually *compute* the projection efficiently.

1.1(F) The Hybrid Methods (the “CQ” Algorithm) V

- **An explicit realization when $C = \mathcal{H}$** (He et al., Eq. 1.18): defines $u_n = \frac{1}{2}(x_n + y_n)$ and half-spaces $C_n = \{z : \langle x_n - u_n, z - u_n \rangle \leq 0\}$, $Q_n = \{z : \langle x_0 - x_n, z - x_n \rangle \leq 0\}$, for which $P_{C_n \cap Q_n} x_0$ can be written down in closed form (via two auxiliary points z_n, w_n built from inner products of the current data) – avoiding any numerical optimization subproblem entirely.
- **A further variant for general C** (He–Yang, Eq. 1.19): combines Eq. 1.15's C_n, Q_n with an *extra* projection back onto C at the end, $u_{n+1} = P_{C_n \cap Q_n} x_0$, $x_{n+1} = P_C u_{n+1}$, which is sometimes easier to compute when C has simple structure (e.g., a ball or a box) even though $C_n \cap Q_n$ does not.

1.1(F) The Hybrid Methods (the “CQ” Algorithm) VI

Takeaway for this section. We now have an entire toolbox of schemes – Picard, Krasnosel'skiĭ, (Krasnosel'skiĭ–)Mann, Ishikawa, Halpern, viscosity, regularized, and hybrid/CQ – trading off simplicity, computational cost per step, and the strength of convergence (weak vs. strong) they guarantee. Chapters 3–7 of the book dig deeply into refinements of exactly this family.

1.2 Overview: Many Problems Are Secretly Fixed Point Problems

The reason the fixed point problem (Eq. 1.1) matters so much is that a huge variety of classical problems – nonlinear equations, equilibrium problems, optimization problems, integral and differential equations – can all be *rewritten* as “find x^* with $T(x^*) = x^*$ ” for a suitable operator T . Once rewritten, every iteration scheme from Section 1.1 becomes available to solve them.

Our running example is already an instance of this

Finding the projection $P_C(x_0)$ of a point x_0 onto a closed convex set C (as in $C = \text{unit disk}$, $x_0 = (3, 4)$) is *by definition* the fixed point problem for $T = P_C$: any $x^* \in C$ with $P_C(x_0)$ landing back on itself. Since $\text{Fix}(P_C) = C$ always, “is x_0 (or its projection) in C ?” is exactly a tiny *convex feasibility problem*: find some point in the (nonempty, closed, convex) set C . We revisit this explicitly after introducing variational inequalities below.

1.2(A) The Nonlinear Equation I

Nonlinear equation (Eq. 1.20)

$$F(x) = 0,$$

where $F : C \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a given operator.

- Finding roots of F is fundamental in numerical mathematics; in general one cannot solve Eq. 1.20 directly and must resort to iterative methods.

The trick: rewrite Eq. 1.20 *equivalently* as the fixed point problem $T(x) = x$ for some operator T built from F (called the *iteration function*). For real, single-variable F , the most popular choice is Newton's method:

1.2(A) The Nonlinear Equation II

Newton's iteration function (Eq. 1.21)

$$T(x) = x - \frac{F(x)}{F'(x)}.$$

- A fixed point of T (where $x = T(x)$) forces $F(x)/F'(x) = 0$, i.e. $F(x) = 0$ (assuming $F'(x) \neq 0$) – exactly a root of F .
- Running Picard iteration on *this* T is precisely Newton's method for root-finding.

1.2(A) The Nonlinear Equation III

Theorem 1.5: a whole family of higher-order methods

Set $F_1 = F$ and, recursively for $m \geq 2$,

$$F_m(x) := \frac{F_{m-1}(x)}{[F'_{m-1}(x)]^{1/m}}.$$

Then

$$G_m(x) = x - \frac{F_{m-1}(x)}{F'_{m-1}(x)}$$

defines an iteration function whose order of convergence, for simple roots, is m .

- $m = 2$: $G_2 = T$ from Eq. 1.21 (Newton's method, order 2).
- $m = 3$: $G_3(x) = x - \frac{F(x)F'(x)}{F'^2(x) - \frac{1}{2}F(x)F''(x)}$ – this is the iteration function of *Halley's method* (cubic, i.e. order-3 convergence).

Python: Newton's Method as a Fixed Point Iteration

We solve $F(x) = x^2 - 2 = 0$ (root $\sqrt{2}$) by running Picard iteration on Newton's iteration function $T(x) = x - F(x)/F'(x)$ from Eq. 1.21.

```
1 def F(x): return x**2 - 2
2 def Fp(x): return 2*x
3
4 def T_newton(x):                # Eq. 1.21: Newton's iteration function
5     return x - F(x) / Fp(x)
6
7 x = 1.0                        # x0, arbitrary starting guess
8 for n in range(6):
9     print(f"x_{n} = {x:.10f}, F(x_{n}) = {F(x):.2e}")
10    x = T_newton(x)             # Picard iteration on T = Newton's map
11 print(f"x_6 = {x:.10f} (true sqrt(2) = {2**0.5:.10f})")
```

Output (excerpt): $x_0 = 1.0000000000$, $x_1 = 1.5000000000$, $x_2 = 1.4166666667$, $x_3 = 1.4142156863$, $x_4 = 1.4142135624$ – already correct to 9 decimal digits by step 4, illustrating the quadratic convergence rate promised by Theorem 1.5 with $m = 2$.

1.2(B) The Equilibrium Problem I

Many classical existence results predate, and motivate, the modern equilibrium-problem framework.

Theorem 1.6 (Hartman–Stampacchia, 1966)

Let C be a compact convex subset of $\mathbb{R}^n$ and $f : C \rightarrow \mathbb{R}^n$ continuous. Then there exists $x_0 \in C$ such that

$$\langle f(x_0), y - x_0 \rangle \geq 0 \quad \text{for all } y \in C.$$

Theorem 1.7 (Ky Fan's best approximation theorem, 1969)

Let C be a nonempty compact convex set in a normed vector space E . Then, for any continuous $f : C \rightarrow E$, there exists $x_0 \in C$ such that

$$\|x_0 - f(x_0)\| = \min_{y \in C} \|y - f(x_0)\|.$$

In particular, if $f(C) \subset C$, then x_0 is a fixed point of f .

1.2(B) The Equilibrium Problem II

These two results underlie variational inequalities, complementarity problems, optimization, and game theory.

1.2(B) The Equilibrium Problem III

The equilibrium problem, EP (Blum–Oettli, 1994)

Let E be a normed vector space, $C \subseteq E$ nonempty convex, and $f : C \times C \rightarrow \mathbb{R}$ a bifunction (a function of two arguments). The *equilibrium problem* is:

Find $x_0 \in C$ such that $f(x_0, y) \geq 0$ for all $y \in C$.

- $f(x, y)$: a “comparison” between candidate points $x, y \in C$; intuitively, x_0 is an equilibrium if no y “improves on” x_0 .

Standing assumptions on f (for the Hilbert-space theory below), for all $x, y, z \in C$:

- (a1) $f(x, x) = 0$;
- (a2) $f(x, y) + f(y, x) \leq 0$ (*monotonicity*);
- (a3) $\lim_{t \rightarrow 0} f(zt + (1 - t)x, y) \leq f(x, y)$;
- (a4) $y \mapsto f(x, y)$ is convex and lower semicontinuous, for each fixed x .

1.2(B) The Equilibrium Problem IV

Lemma 1.1 (existence of an approximate equilibrium)

Let f satisfy (a1)–(a4), $r > 0$, $x \in \mathcal{H}$. Then there exists $z \in C$ such that

$$f(z, y) + \frac{\langle y - z, z - x \rangle}{r} \geq 0 \quad \text{for all } y \in C.$$

Lemma 1.2 (the resolvent T_r of an equilibrium bifunction)

Under (a1)–(a4), for $r > 0$ and $x \in \mathcal{H}$, define

$$T_r(x) = \left\{ z \in C : f(z, y) + \frac{\langle y - z, z - x \rangle}{r} \geq 0 \text{ for all } y \in C \right\}.$$

Then: (1) T_r is single-valued; (2) T_r is firmly nonexpansive (reminder:

$\|T_r(x) - T_r(y)\|^2 \leq \langle T_r(x) - T_r(y), x - y \rangle$); (3) $\text{Fix}(T_r) = EP(f)$; (4) $EP(f)$ (the solution set of the equilibrium problem) is nonempty, closed, and convex.

1.2(B) The Equilibrium Problem V

Why this matters for us: Lemma 1.2 says solving the equilibrium problem is *literally* a fixed point problem for the (firmly nonexpansive!) resolvent operator T_r – so every scheme from Section 1.1 (Krasnosel'skiĭ–Mann, Halpern, viscosity, hybrid methods, . . .) applies directly.

1.2(B') Special Cases of the Equilibrium Problem I

The equilibrium problem is a common “meta-format” for several well-known problems: pick the right bifunction f , and each becomes a special case of EP (and hence of the fixed point problem, via T_r).

(1) The minimization problem (MP)

Find $x \in C$ such that $\phi(x) \leq \phi(y)$ for all $y \in C$, where $\phi : C \rightarrow \mathbb{R}$.

- Set $f(x, y) = \phi(y) - \phi(x)$: then $\text{EP} \equiv \text{MP}$ exactly (an equilibrium is precisely a global minimizer of ϕ).

(2) The saddle point problem (SPP)

Find $(\bar{x}_1, \bar{x}_2) \in C_1 \times C_2$ such that $L(\bar{x}_1, y_2) \leq L(\bar{x}_1, \bar{x}_2) \leq L(y_1, \bar{x}_2)$ for all $(y_1, y_2) \in C_1 \times C_2$, where $L : C_1 \times C_2 \rightarrow \mathbb{R}$.

- Set $C = C_1 \times C_2$ and $f((x_1, x_2), (y_1, y_2)) = L(y_1, x_2) - L(x_1, y_2)$: then $\text{EP} \equiv \text{SPP}$.

1.2(B') Special Cases of the Equilibrium Problem II

(3) The Nash equilibrium problem (NEP)

Let $I = \{1, \dots, n\}$ be n players, C_i the strategy set of player i , $C = \prod_{i \in I} C_i$, and $\phi_i : C \rightarrow \mathbb{R}$ the loss of player i . Writing $x^i = (x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n)$ for “everyone’s strategy except player i ’s,” find $\bar{x} \in C$ such that, for every $i \in I$, $\phi_i(\bar{x}) \leq \phi_i(\bar{x}^i, y_i)$ for all $y_i \in C_i$ (no player can unilaterally improve their loss).

- Set $f(x, y) = \sum_{i=1}^n (\phi_i(x^i, y_i) - \phi_i(x))$: then EP $\equiv$ NEP.

(4) The fixed point problem itself (FPP)

Find $x_0 \in C$ such that $\phi(x_0) = x_0$, $\phi : C \rightarrow C$ (this is Eq. 1.1, restated).

- Set $f(x, y) = \langle x - \phi(x), y - x \rangle$: then x_0 solves FPP iff it solves EP. (This closes the loop: fixed points are themselves a special equilibrium problem, and equilibria are solved via fixed points of T_r – the two notions are deeply intertwined.)

1.2(B') Special Cases of the Equilibrium Problem III

(5) The variational inequality problem (VIP)

Let E be a normed vector space with dual E^* , $C \subseteq E$ convex, and $\phi : C \rightarrow E^*$. Find $x_0 \in C$ such that

$$\langle \phi(x_0), y - x_0 \rangle \geq 0 \quad \text{for all } y \in C.$$

- Set $f(x, y) = \langle \phi(x), y - x \rangle$: then $\text{VIP} \equiv \text{EP}$.

(6) The nonlinear complementarity problem (NCP)

Let $C \subseteq E$ be a closed convex cone and $C^* = \{x^* \in E^* : \langle x^*, y \rangle \geq 0 \text{ for all } y \in C\}$ its polar cone. Find $x_0 \in C$ such that $\phi(x_0) \in C^*$ and $\langle \phi(x_0), x_0 \rangle = 0$.

1.2(B') Special Cases of the Equilibrium Problem IV

Remark 1.1: all these problems coincide

(a) $\text{VIP} \equiv \text{NCP}$.

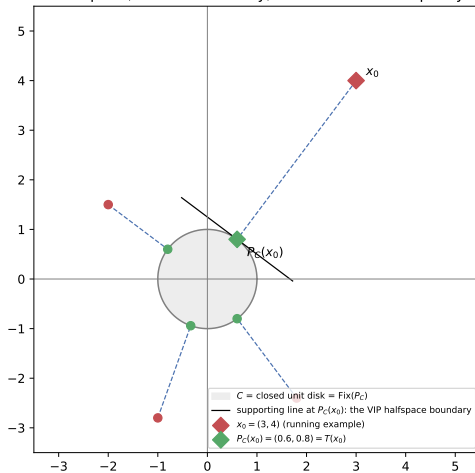
(b) With $f(x, y) = \langle \phi(x), y - x \rangle$, the problems VIP, NCP, and EP are all equivalent.

The book also mentions the *dual equilibrium problem* (including the *Minty-type variational inequality*) as further special cases; we will not need their details here.

Back to our running example. $P_C(x_0)$ is characterized *exactly* by a VIP: $x^* = P_C(x_0)$ if and only if $x^* \in C$ and $\langle x_0 - x^*, y - x^* \rangle \leq 0$ for all $y \in C$ – i.e., every point of C makes a non-acute angle with the vector from x^* back to x_0 . This is exactly the tangent (supporting) line drawn in the figure below at $(0.6, 0.8)$.

1.2(B') Special Cases of the Equilibrium Problem V

$P_C(x_0)$ solves three equivalent problems:
fixed point, convex feasibility, and variational inequality



1.2(B') Special Cases of the Equilibrium Problem VI

The projection $P_C(x_0)$ solves three equivalent problems at once: the **fixed point problem** for $T = P_C$ (green points are all fixed, since $\text{Fix}(P_C) = C$); the (trivial) **convex feasibility problem** of finding a point in C ; and the **variational inequality** shown by the supporting line at $P_C(x_0)$, separating x_0 from the rest of C .

1.2(C) The Hierarchical Fixed Point Problem

A refinement: among possibly *many* fixed points of T , select the one closest to (or otherwise “preferred by”) a second nonexpansive map V .

Hierarchical variational inequality problem, HVIP

Let $T, V : C \rightarrow C$ be nonexpansive. Find $x^* \in \text{Fix}(T)$ such that

$$\langle x^* - Vx^*, y - x^* \rangle \geq 0 \quad \text{for all } y \in \text{Fix}(T).$$

Equivalently, $x^* = P_{\text{Fix}(T)} Vx^*$ – i.e., x^* is a fixed point of the (again nonexpansive) map $P_{\text{Fix}(T)} V$, where P_K denotes metric projection onto a closed convex set K .

- **If $V = \text{Id}$:** the HVIP solution set is exactly $\text{Fix}(T)$ (no extra selection is imposed).
- **If $V \equiv u$ (a constant map):** HVIP becomes “find the fixed point of T closest to u ,”
 $x^* = P_{\text{Fix}(T)} u = \operatorname{argmin}_{x \in \text{Fix}(T)} \frac{1}{2} \|u - x\|^2$ – exactly the target of the Halpern iteration (Section 1.1(D))!
- **As an equilibrium problem:** with $C = \text{Fix}(T)$ and $f(x, y) = \langle (\text{Id} - V)x, y - x \rangle$, HVIP is exactly EP for this f .

1.2(D) The Integral Equation I

Integral and integro-differential equations form another large class of operator equations naturally reformulated as fixed point problems.

A general integral equation (Eq. 1.22)

$$y(x) = f(x) + \int_0^1 K(x, s, y(x), y(s)) ds \quad \text{for all } x \in [0, 1].$$

- K : the *kernel* of the equation; f : a given “source” function; y : the unknown function.

Three important special cases:

- **Radiation transfer:** $y(x) = 1 + \int_0^1 \frac{s y(s) y(x)}{s+x} \varphi(s) ds$, with φ given.
- **Urysohn equation:** $y(x) = 1 + \int_0^1 K(x, s, y(s)) ds$.
- **Nonlinear Fredholm integral equation** (Eq. 1.23): $y(x) = f(x) + \lambda \int_0^1 K(x, s, y(s)) ds$, with $\lambda \in \mathbb{R}$ given.

1.2(D) The Integral Equation II

Turning Eq. 1.23 into a fixed point problem

Assumptions: $K : [0, 1] \times [0, 1] \times I \rightarrow \mathbb{R}$ continuous and bounded; K is L -Lipschitz in its third variable ($|K(x, s, z_1) - K(x, s, z_2)| \leq L|z_1 - z_2|$); $f : [0, 1] \rightarrow \mathbb{R}$ continuous; $\lambda \in \mathbb{R}$ given; the unknown $\varphi : [0, 1] \rightarrow I$ is continuous.

Let $X = C[0, 1]$ with the Chebyshev metric $d(\varphi_1, \varphi_2) = \max_{x \in [0, 1]} |\varphi_1(x) - \varphi_2(x)|$ (a complete metric space), and define, for $\varphi \in X$,

$$(T\varphi)(x) = f(x) + \lambda \int_0^1 K(x, s, \varphi(s)) ds \quad (\text{Eq. 1.24}).$$

- T maps X into itself (continuity of K , f gives continuity of $T\varphi$), so Eq. 1.23 is equivalent to the fixed point problem $\varphi = T(\varphi)$.

1.2(D) The Integral Equation III

- One can show T is $L|\lambda|$ -Lipschitz continuous.
- **Choosing** $|\lambda| < 1/L$ makes T a *strict contraction*, so the Banach contraction principle gives a unique fixed point – the unique solution of Eq. 1.23 – obtainable via plain **Picard iteration**.
- This is a case where the classical (contraction-based) theory already suffices; the nonexpansive machinery of Section 1.1 becomes essential only when such a Lipschitz constant is not small enough to guarantee $|\lambda|L < 1$.

1.2(D) The Integral Equation IV

The Volterra integral equation of the second kind (Eq. 1.25)

$$y(x) = f(x) + \lambda \int_a^x K(x, s, y(s)) ds \quad \text{for all } x \in [a, b].$$

If K is Lipschitz in its third variable, Eq. 1.25 has a unique continuous solution. Defining

$$(T\varphi)(x) = f(x) + \lambda \int_a^x K(x, s, \varphi(s)) ds \quad (\text{Eq. 1.26}),$$

Eq. 1.25 becomes the fixed point problem $\varphi = T(\varphi)$; for suitable parameters, T is again a contraction, and Picard iteration approximates the unique solution.

1.2(E) The Ordinary Differential Equation I

Even ODEs reduce to (Volterra) integral equations, hence to fixed point problems.

Initial value problem, IVP (Eq. 1.27)

$$\begin{cases} y' = f(x, y), \\ y(x_0) = y_0. \end{cases}$$

Equivalent Volterra integral equation:

$$y(x) = y_0 + \int_{x_0}^x f(s, y(s)) ds.$$

1.2(E) The Ordinary Differential Equation II

Two-point boundary value problem, BVP (Eq. 1.28)

$$\begin{cases} y'' = f(x, y), \\ y(a) = A, \quad y(b) = B. \end{cases}$$

Equivalent integral form:

$$y(x) = \frac{x-a}{b-a}B + \frac{b-x}{b-a}A - \int_a^b G(x,s)f(s,y(s))ds.$$

1.2(E) The Ordinary Differential Equation III

The Green's function for the BVP (Eq. 1.29)

$$G(x, s) = \begin{cases} \frac{(s-a)(b-x)}{b-a}, & a \leq s \leq x \leq b, \\ \frac{(x-a)(b-s)}{b-a}, & a \leq x \leq s \leq b, \end{cases}$$

associated with the homogeneous problem $y'' = 0$, $y(a) = 0$, $y(b) = 0$.

- $G(x, s)$: encodes “how much a unit forcing term at location s influences the solution’s value at location x ,” for the simplest (homogeneous) version of the boundary value problem.

1.2(E) The Ordinary Differential Equation IV

- Under standard assumptions on f (continuous, Lipschitz in the last variable), the corresponding integral operators satisfy a contractive condition, so the Banach fixed point theorem – and Picard iteration – again apply directly.
- **Big picture for Section 1.2:** nonlinear equations, equilibrium/variational/game-theoretic problems, integral equations, and differential equations all reduce to the *same* underlying computational primitive: find x^* with $T(x^*) = x^*$. This is precisely why the iterative schemes of Section 1.1 are so broadly useful.

Python: Convex Feasibility via the Running Example's VIP Characterization

We numerically confirm that $P_C(x_0)$ (our running example) satisfies the variational-inequality characterization from Section 1.2(B')(5): $\langle x_0 - x^*, y - x^* \rangle \leq 0$ for every $y \in C$.

```
import numpy as np
def project_disk(x):
    n = np.linalg.norm(x)
    return x if n <= 1.0 else x / n

x0 = np.array([3.0, 4.0])
x_star = project_disk(x0)           # x* = P_C(x0) = (0.6, 0.8)
print("x* = P_C(x0) =", x_star)

rng = np.random.default_rng(0)     # sample y in C, check the VIP
worst = -np.inf
for _ in range(200_000):
    y = rng.uniform(-1, 1, size=2)
    if np.linalg.norm(y) > 1:       # keep only y in C
        continue
    val = np.dot(x0 - x_star, y - x_star)  # should be <= 0 always
    worst = max(worst, val)
print(f"max <x0-x*, y-x*> over sampled y in C = {worst:.6f} (should be <= 0)")
```

Output (excerpt): $x^* = [0.6 \ 0.8]$, and $\max \dots = 0.000000$ (attained only at $y = x^*$ itself) – confirming numerically that $P_C(x_0)$ solves the VIP, hence the fixed point problem for $T \equiv P_C$. hence^{66/71}

1.3 Outline: Roadmap of the Book I

Since its introduction in 1953, the Krasnosel'skiĭ–Mann iteration has been a cornerstone of fixed point theory, with an enormous and growing literature on its convergence in different spaces and its many variants. Especially in the last decade, attention has focused on it as a key tool in optimization, together with accelerations such as *inertia* and *relaxation*. Here is how the rest of the book is organized.

Chapter 3: The original Krasnosel'skiĭ–Mann iteration

The mean iteration scheme introduced by Mann, recently extended to a broad class of relaxed fixed point algorithms on the convex hull (or the affine hull) of orbits. Covers Krasnosel'skiĭ–Mann iterations with *perturbations* and their bounded perturbation resilience, and summarizes recent research on convergence *rate*, including global pointwise and ergodic iteration-complexity bounds.

1.3 Outline: Roadmap of the Book II

Chapter 4: Relations to operator splitting methods

Shows how several famous operator-splitting methods can be recast as instances of the Krasnosel'skii–Mann iteration: the proximal point method, the forward-backward splitting method, the backward-forward splitting method, the Douglas–Rachford splitting method, the Davis–Yin splitting method, and a class of primal-dual splitting methods.

Chapter 5: The inertial Krasnosel'skii–Mann iteration

Recent progress on the inertial variant first introduced by Maingé, especially the *general* inertial Krasnosel'skii–Mann iteration. Focuses on the inertial parameters and presents several valid ranges for them; also covers the alternated inertial and the online inertial Krasnosel'skii–Mann iterations.

1.3 Outline: Roadmap of the Book III

Chapter 6: The multi-step inertial Krasnosel'skiĭ–Mann iteration

Presents its convergence analysis, including two classes of inertial parameter sequences that do not involve the iterative sequence itself. Introduces a general Krasnosel'skiĭ–Mann iteration on the affine hull of orbits and a modified Krasnosel'skiĭ–Mann iteration. Applications: multi-step inertial versions of the forward-backward, backward-forward, Douglas–Rachford, Davis–Yin, and primal-dual splitting methods.

Chapter 7: Choosing the relaxation parameters

Discusses the line search method, the online choice, and the optimal choice of relaxation parameters. Using the equivalence between fixed point problems and variational inequality problems, presents (multi-step) inertial Krasnosel'skiĭ–Mann iterations with *adaptive* relaxation parameters, and a residual algorithm exploiting the equivalence between fixed point problems and nonlinear monotone equations.

1.3 Outline: Roadmap of the Book IV

Chapter 8 (final): Two applications

Asynchronous parallel coordinate-update methods, and cyclic coordinate-update algorithms. Convergence analysis and convergence rate are presented under suitable conditions for both.

In short: Chapter 1 (this chapter) built the vocabulary and the basic toolbox of iteration schemes; Chapters 3–8 progressively refine, accelerate, connect, and apply the Krasnosel'skiĭ–Mann iteration across the rest of the book. (Chapter 2, not previewed above, lays the formal groundwork – precise definitions of contractions, monotonicity, and related operator classes referenced throughout this chapter.)

Chapter 1 — Key Takeaways

- The **fixed point problem** ($T(x^*) = x^*$ for $T : C \rightarrow C$) unifies a remarkable range of computational questions; this chapter's job is cataloguing *how to compute* such an x^* .
- **Picard iteration** ($x_{n+1} = T(x_n)$) converges for contractions, but can fail for merely nonexpansive maps (our $T = -\text{Id}$ example: oscillates forever).
- **Krasnosel'skiĭ averaging** ($x_{n+1} = \frac{1}{2}x_n + \frac{1}{2}T(x_n)$, generalized to Krasnosel'skiĭ–Mann with λ_n) repairs this: on our running example ($T = P_C$, $x_0 = (3, 4)$) it converges geometrically, $r_0=5, r_1=3, r_2=2, \dots, r_6=1.0625 \rightarrow 1$.
- **Stronger convergence** (strong, not just weak) is bought via anchoring (**Halpern**), moving anchors (**viscosity**), regularization, or projections (**hybrid/CQ methods**) – each with its own trade-offs in conditions and computational cost.
- **Section 1.2's big idea**: nonlinear equations (Newton's method), equilibrium problems (which subsume minimization, saddle points, Nash equilibria, variational inequalities, and complementarity problems), and integral/differential equations all reduce to the *same* fixed point primitive – and our running example's projection $P_C(x_0)$ instantiates this concretely as a fixed point / convex feasibility / variational inequality problem all at once.
- The rest of the book (Chapters 2–8) builds outward from these definitions: formal groundwork (Ch. 2), the Krasnosel'skiĭ–Mann iteration in depth and its splitting-method relatives (Ch. 3–4), inertial acceleration (Ch. 5–6), adaptive relaxation (Ch. 7), and parallel/cyclic coordinate-update applications (Ch. 8).